MURILO CAVAZZANA RIBEIRO

ATIVIDADE AVALIATIVA

PROJETO E DESENVOLVIMENTO DE SOFTWARE II

Londrina
2026

MURILO CAVAZZANA RIBEIRO

ATIVIDADE AVALIATIVA

PROJETO E DESENVOLVIMENTO DE SOFTWARE II

Atividade avaliativa apresentada ao Curso de Engenharia de Software do Centro Universitário Filadélfia - UniFil, necessária para a aprovação na disciplina Projeto e Desenvolvimento de Software II.

Orientador: Eron Ponce Pereira

Londrina
2026

SUMÁRIO

1. INTRODUÇÃO 1

2. DESENVOLVIMENTO 1

2.1 STACK UTILIZADA 1

2.2 REQUISITOS FUNCIONAIS 2

2.3 REQUISITOS NÃO FUNCIONAIS 3

2.4 DIAGRAMA DE CASO DE USO 3

2.5 DESCRIÇÃO DOS CASOS DE USO 4

2.6 DIAGRAMA ENTIDADE RELACIONAMENTO (DER) 4

2.7 ARQUITETURA 5

2.8 TESTES 6

3. CONCLUSÃO 6

REFERÊNCIAS 7

1. INTRODUÇÃO

O Sistema de Horários é uma aplicação web para montar e manter a grade de horários das turmas da UniFil. O projeto busca facilitar o trabalho de criação e organização das grades, que é feito à mão em planilhas (Excel e Google Docs), o que é lento, gera várias versões do mesmo arquivo e permite erros como um professor alocado em duas turmas no mesmo horário.

O sistema centraliza os cadastros de professores, unidades curriculares (UCs) e turmas, e oferece uma grade semanal por turma onde o coordenador escolhe, para cada dia e bloco de aula, qual UC será dada e por qual professor. O próprio banco de dados impede conflitos de horário.

2. DESENVOLVIMENTO

Nesta seção, detalha-se a execução prática do projeto, documentando os requisitos funcionais e não funcionais, casos de uso, Diagrama entidade-relacionamento (DER), arquitetura e testes.

2.1. STACK UTILIZADA

Camada	Tecnologia	Motivo
Front-end	Next.js 15, React 19, TypeScript, Tailwind CSS	Roteamento por pastas, componentes reutilizáveis, estilo rápido
Back-end	NestJS 11, TypeScript	Estrutura em módulos (controller, service, DTO) com validação nativa
Banco	SQLite, acessado pelo Prisma ORM	Relacional, sem instalação de servidor, migrations versionadas
Autenticação	JWT (JSON Web Token) com senha em bcrypt	Padrão para APIs REST sem sessão no servidor

Toda a aplicação é escrita em TypeScript, tanto no front quanto no back, o que permite usar o mesmo vocabulário de tipos nas duas partes.

2.2. REQUISITOS FUNCIONAIS

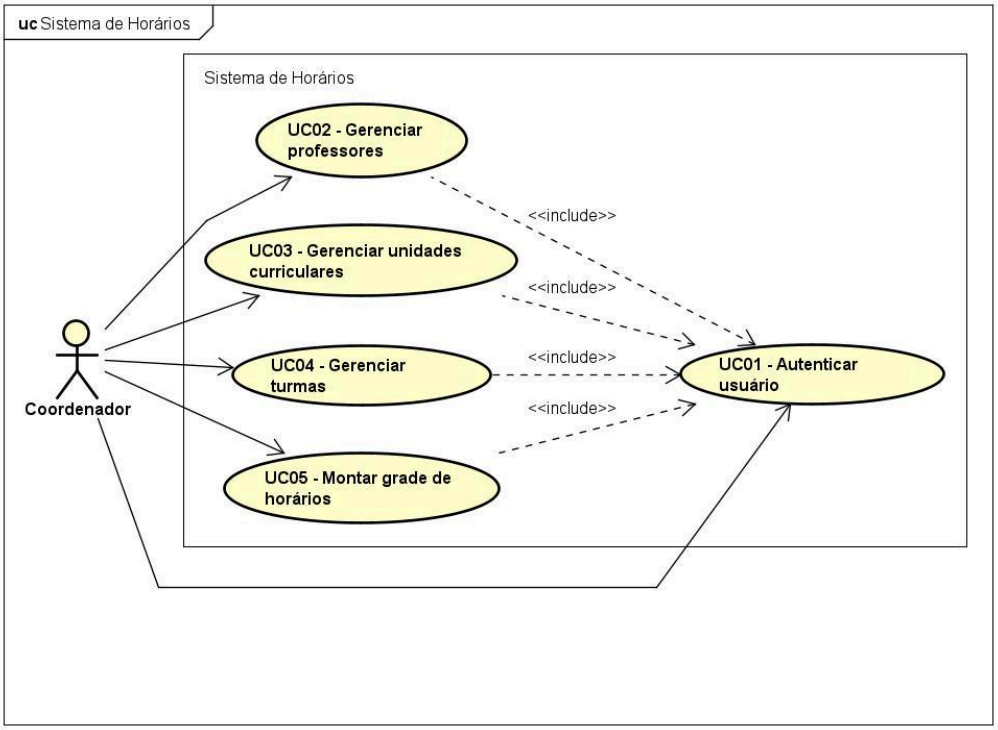
Código	Requisito
RF01	O sistema deve permitir que o usuário se autentique com e-mail e senha.
RF02	O sistema deve bloquear o acesso a todas as funcionalidades para usuários não autenticados.
RF03	O sistema deve permitir cadastrar, listar, editar e excluir professores.
RF04	O sistema deve permitir cadastrar, listar, editar e excluir unidades curriculares (nome, código e carga horária).
RF05	O sistema deve permitir cadastrar, listar, editar e excluir turmas (nome e período).
RF06	O sistema deve exibir a grade semanal de uma turma (segunda a sábado, dois blocos de aula por dia).
RF07	O sistema deve permitir alocar, alterar e remover uma aula (UC + professor) em cada posição da grade.
RF08	O sistema deve impedir que uma turma tenha duas aulas no mesmo dia e bloco.
RF09	O sistema deve impedir que um professor seja alocado em duas turmas no mesmo dia e bloco.
RF10	O sistema deve permitir imprimir ou salvar em PDF a grade de uma turma.
RF11	Ao excluir um professor, UC ou turma, as aulas ligadas a ele devem ser removidas da grade.

2.3. REQUISITOS NÃO FUNCIONAIS

Código	Requisito
RNF01	As senhas devem ser armazenadas com hash (bcrypt), nunca em texto puro.
RNF02	A comunicação entre front e back deve ser feita por API REST com JSON, protegida por token JWT.
RNF03	Toda entrada do usuário deve ser validada no servidor antes de chegar ao banco.
RNF04	As regras de conflito de horário devem ser garantidas por restrições do próprio banco de dados.
RNF05	A interface deve funcionar em telas de celular e de desktop.
RNF06	O sistema deve rodar localmente apenas com Node.js instalado, sem servidor de banco externo.
RNF07	O código deve ser escrito em TypeScript com verificação de tipos estrita.
RNF08	As mensagens da interface e da API devem estar em português.

2.4. DIAGRAMA DE CASO DE USO

O sistema possui um único ator, o Coordenador, responsável por todos os cadastros e pela montagem da grade.



2.5. DESCRIÇÃO DOS CASOS DE USO

UC01 - Autenticar usuário: O coordenador informa e-mail e senha. O sistema confere o hash da senha e devolve um token JWT válido por 8 horas. Com senha errada, o sistema responde "E-mail ou senha inválidos".

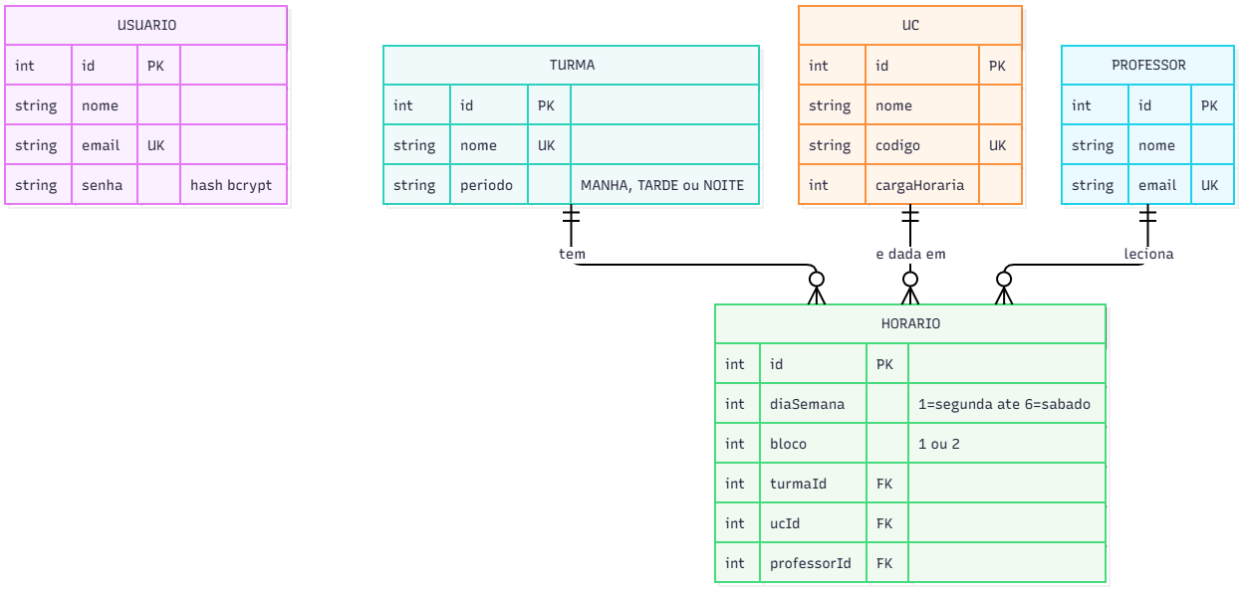
UC02 - Gerenciar professores: O coordenador cadastra um professor com nome e e-mail, consulta a lista, edita os dados ou exclui o registro. O e-mail é único.

UC03 - Gerenciar unidades curriculares: O coordenador cadastra uma UC com nome, código e carga horária em horas, consulta, edita ou exclui. O código é único.

UC04 - Gerenciar turmas: O coordenador cadastra uma turma com nome e período (manhã, tarde ou noite), consulta, edita ou exclui.

UC05 - Montar grade de horários: O coordenador escolhe uma turma e vê a grade semanal. Ao clicar em uma posição vazia, escolhe a UC e o professor da aula. Ao clicar em uma posição ocupada pode trocar a UC ou o professor, ou remover a aula. Fluxos alternativos: se a turma já tem aula naquele dia e bloco, ou se o professor já está em outra turma naquele dia e bloco, o sistema recusa e mostra a mensagem de conflito.

2.6. DIAGRAMA ENTIDADE RELACIONAMENTO (DER)



Restrições de integridade no banco:

- HORARIO (turmaId, diaSemana, bloco) é único: a turma só tem uma aula por posição da grade (RF08).
- HORARIO (professorId, diaSemana, bloco) é único: o professor só dá uma aula por posição da grade (RF09).
- As três chaves estrangeiras de A tabela HORARIO usam ON DELETE CASCADE (RF11).

A tabela USUARIO guarda quem pode entrar no sistema e não se relaciona com as demais: ela existe para a autenticação, não para a grade.

2.7. ARQUITETURA



O front-end nunca acessa o banco. Toda operação passa pela API, que valida os dados (class-validator), verifica o token (guard global) e só então chama o Prisma.

Cada entidade na API segue o mesmo padrão do NestJS: um DTO com as regras de validação, um service com as operações de banco e um controller com as rotas REST (GET, POST, PUT, DELETE)

Rota	Operações
POST /auth/login	Autenticação
/professores , /professores/:id	CRUD de professores
/ucs , /ucs/:id	CRUD de unidades curriculares
/turmas , /turmas/:id	CRUD de turmas
/horarios?turmaId= , /horarios/:id	CRUD de aulas da grade

No front-end, as três telas de cadastro reutilizam um único componente genérico (Crud.tsx), que recebe a lista de campos e a rota da API. A grade de horários é uma tela própria.

2.8. TESTES

A API possui um teste ponta a ponta (api/test/crud.test.ts) que roda contra o servidor em execução e verifica: bloqueio sem token, login certo e errado, criação, edição e exclusão de cada entidade, rejeição de dados inválidos, os dois tipos de conflito da grade e a exclusão em cascata.

3. CONCLUSÃO

O projeto partiu de uma versão anterior em PHP com Laravel, feita em 2025 para a apresentação da primeira banca da disciplina de Projeto de estágio, que tinha os cadastros de disciplina e UC mas não persistia a grade de horários e tinha uma migration referenciando uma tabela inexistente. Reescrever em TypeScript com Next.js e NestJS me fez repensar o modelo de dados antes de escrever código: a regra de negócio central do sistema (ninguém em dois lugares ao mesmo tempo) cabe em duas restrições de unicidade no banco. Isso elimina código de verificação no servidor e garante que a regra vale mesmo que alguém insira dados por outro caminho. Apesar de simples, o sistema funciona muito bem e faz o seu papel.

Outros aprendizados:

- Separar front e back por uma API REST deixa cada parte testável sozinha. O teste ponta a ponta da API não precisa de navegador.
- Os DTOs do NestJS rejeitam dados errados antes de qualquer lógica, e a mensagem de erro chega pronta para a interface.
- A versão anterior tinha “arrastar e soltar” com dados fixos no código. Uma grade clicável que grava no banco entrega mais valor com menos código.
- ORM com migrations (Prisma) mantém o DER e o banco sempre iguais, o que facilitou escrever a documentação a partir do schema.

REFERÊNCIAS

NESTJS. **NestJS Documentation**. Disponível em: https://docs.nestjs.com/. Acesso em: 21 out. 2026.

VERCEL. **Next.js Documentation**. Disponível em: https://nextjs.org/docs. Acesso em: 21 out. 2026.

PRISMA. **Prisma ORM Documentation**. Disponível em: https://www.prisma.io/docs. Acesso em: 21 out. 2026.

TAILWIND LABS. **Tailwind CSS Documentation**. Disponível em: https://tailwindcss.com/docs. Acesso em: 21 out. 2026.